

Web Application Basics & OWASP Top 10

OWASP TOP 10

1. Broken Access Control

- **Description:** Users acting outside of their intended permissions. Modify URL, internal Application State, modify API requests. IDOR based exploit. Metadata manipulation, like JWT. Cross-Origin Resource Sharing (CORS) misconfigs.
- **Prevention:** Deny by Default for CORS, record ownership to prevent from IDOR, expiry of Access Token to expire quickly (adding refresh state, stateless)

2. Security Misconfiguration

- **Description:** Missing Security hardening, unnecessary feature enable, default accounts and passwords, disabling security feature, misconfigs in databases, application server, etc.
- **Prevention:** Easy and periodical security hardening, enable security settings, disable unnecessary features, correct configs in servers, databases, etc, error handling, updating default setups.

3. Software Supply Chain

- **Description:** Software can be vulnerable, unsupported, or out of date. No regular VA. Not upgrading platforms, frameworks, and dependencies. Developers do not test the compatibility of updated, upgraded, or patched libraries.
- **Prevention:** Reduce attack surface by removing unused dependencies. Monitor sources, obtain components from official sources. CI/CD settings, Sandbox area, Developer IDE for tracking changes.

4. Cryptographic Failures

- **Description:** Weak crypto alg, encryption not enforced, lack server certificates, cryptographic error messages exploits.
- **Prevention:** Store sensitive keys in a hardware or cloud HSM, strong crypto alg, proper hash and salting functions, encryption, using secured protocols, avoiding older alg.

5. Injection

- **Description:** Input data not sanitized, non-parameterized calls.
- **Prevention:** Use a safe API, parameterized, Object relational mapping(ORM)

6. Insecure Design

- **Description:** concerning confidentiality, integrity, availability, and authenticity, lack of proper development lifecycle.
- **Prevention:** Secure development lifecycle, model the threats, test misuses

7. Authentication

- **Description:** Default weak passwords, lack of MFA, reusing the same sessions, non-hash, non-encrypted
- **Prevention:** Strong passwords, with hash and encryption alg, limiting the reuse of sessions

8. Software or Data Integrity Failures

- **Description:** Insecure CI/CD pipelines, downloading updates without verifying integrity verification.
- **Prevention:** Verify digital signs, proper segregation of CI/CD pipelines, configurations

9. Security Logging and Alerting Failures

- **Description:** inconsistent log of data, backups, alert systems, lack differentiate of false positives, true negatives, mishandling errors
- **Prevention:** consistent logs and backups of data, adopt incident response cycles, using well defined detection and alerting tools (IDS, IPS)

10. Mishandling of Exceptional Conditions

- **Description:** failure in handling exceptional conditions, prevention, detection, response. Poor input validation, inconsistent, resource issues
- **Prevention:** using rollbacks incase of failed process, layer of defence, alerting, noise reduction, sanitization, etc_

PORTSWIGGER ACCESS CONTROL

User role can be modified in user profile (Lab 4)

Task: Solve the lab by accessing the admin panel and using it to delete the user “carlos”, accessible only by “roleid = 2” in “/admin”

Vulnerability: Broken Access Control – Vertical Privilege Escalation (as normal user gets access of admin level)

Exploit:

- Logged in with my given credentials, updated the e-mail id.
- Connected with BurpSuite using FoxyProxy.
- In BurpSuite, at Proxy, HTTP History, analyzed for the POST request of email change, sent it to the Repeater.
- In Response side, roleid = 1, so in Request side added “roleid” = 2 in the JSON format given, then Send.
- Refreshed the lab, got access to the admin panel, deleted the user “carlos”.

Solution:

- using ORM with least privilege
- admin verifications (authentications)
- server side protected session handling
- no direct intake of JSON object.

The screenshot displays a web browser window on the left and the Burp Suite interface on the right. The browser shows the 'WebSecurity Academy' page with a 'LAB Solved' badge and a message 'User role can be modified in user profile'. Below this, it says 'Congratulations, you solved the lab!' and 'User deleted successfully!'. The 'Users' section lists 'wiener' with a 'Delete' link. The Burp Suite interface shows a 'Repeater' tab with a list of requests. The selected request is a POST to '/my-account/change-email' with a JSON body containing 'email': 'wiener@gmail.com' and 'roleid': 2. The response shows a 200 status and a JSON body with 'username': 'wiener', 'email': 'wiener@gmail.com', 'apikey': '0Hz1lp85Abp25CydcVjog6eLMWS2ug16', and 'roleid': 2.